University of Bremen
Department of Computer Science
Prof. Dr. h.c. Michael Beetz, PhD

AICOR Institute for Artificial Intelligence
WiSe 2026/2027
Worksheet 3

Exercises for the Lecture Advanced Methods of AI

Asking, Describing, Constraining, Inferring

Tutorial: Thursday, 12.11.2026, 8:30–10:00, room 0.30, ECO 5

Work in pairs with one laptop: open the notebook `03-pizza_queries.ipynb` on Binder (link on Stud.IP). Parts marked **PAPER** are done on this sheet, **NOTEBOOK** in the notebook. Before every **PREDICT**, commit to an answer on this sheet *before* you run the cell.

PROLOGUE

Thursday evening, 19:30

≈ 5 min

> *A small pizzeria in Bremen. A service robot works between the counter and the tables. Seven pizzas are on the counter, three tables have ordered, and two guests must not eat certain things. The robot knows all of this as objects: the classes of `pizza.owl` from last week, plus guests and orders.*

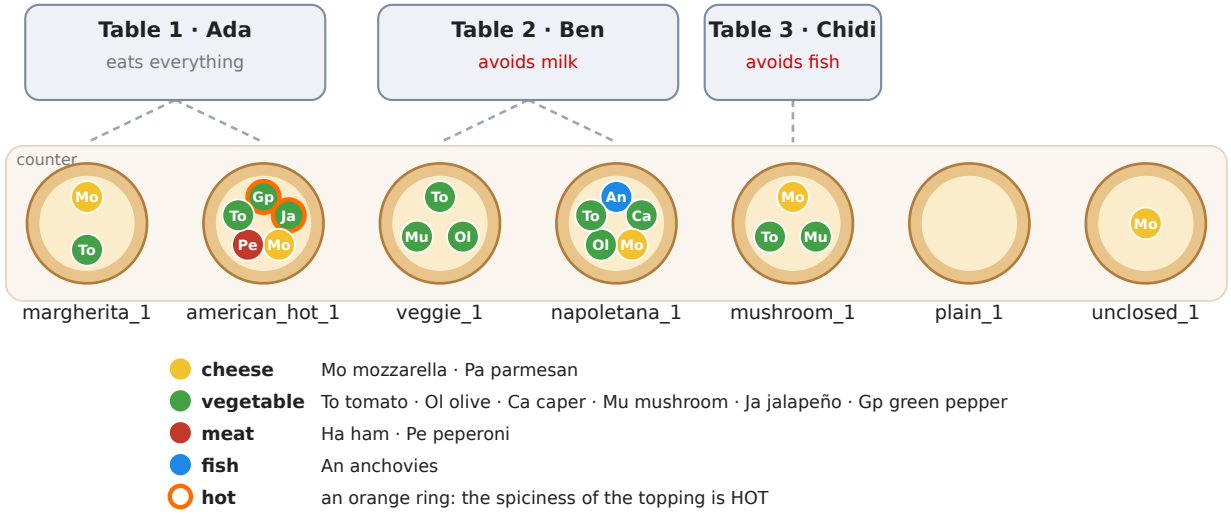


Figure 1. What the robot knows at 19:30. Lines connect each table to the pizzas it ordered. You will draw on this figure all morning: keep a pen ready.

Everything the robot does tonight starts with the same kind of expression, a **description**. In EQL, the query language of KRROOD, "a pizza with a cheese topping" is written as follows; it is the concept of Worksheet 2 below it:

```
a(Pizza)(toppings=a(CheeseTopping)())
```

$$\texttt{Pizza} \sqcap \exists \texttt{hasTopping.CheeseTopping}$$

What the robot *does* with a description is what this tutorial is about. There are five things, one per act:

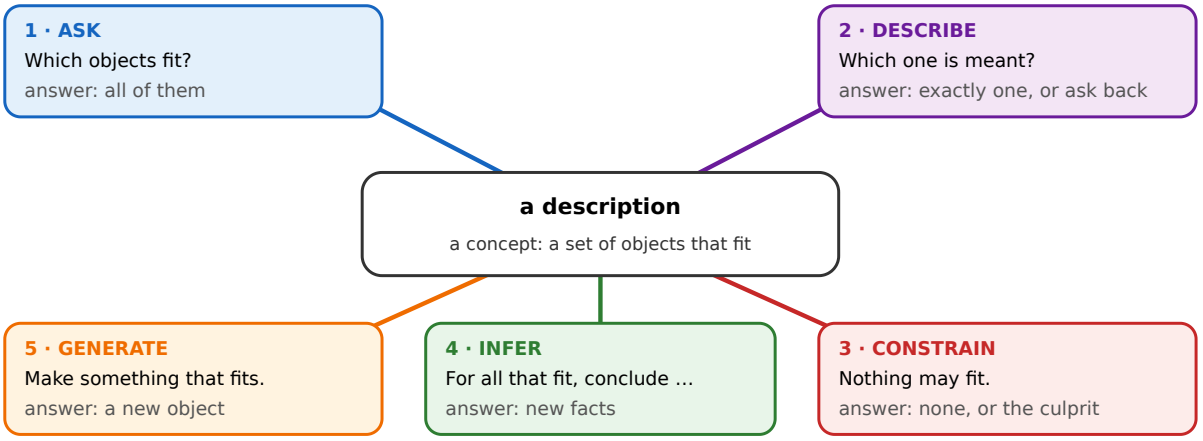


Figure 2. One description, five uses. The expression stays the same; what changes is where the candidates come from and what is done with the objects that fit.

? **Warm-up.** Figure 1 is an *interpretation* in the sense of Worksheet 2. What is its domain? What is the extension of `CheeseTopping`, and of `hasTopping`?

ACT 1 · ASKING

"Which pizzas have cheese?"

≈ 25 min

> *The chef calls across the kitchen. The cheesy pizzas go back into the oven for a minute, so that the cheese melts. The robot has to find them.*

(a) PAPER **A description is a concept.** Write `D = a(Pizza)(toppings=a(CheeseTopping)())` as a first-order formula with one free variable x. PEN Ring the pizzas of Figure 1 that are answers. NOTEBOOK Run Act 1(a) and compare.

Three languages, one meaning.

Description logic	First-order logic	EQL
`Pizza`	`Pizza(p)`	`p = variable(Pizza, domain=pizzas)`
a filler of hasTopping	`hasTopping(p, t)`	`t = flat_variable(p.toppings)`
`∃hasTopping.C`	`∃t (hasTopping(p, t) ∧ C(t))`	`exists(t, HasType(t, C))`, or `toppings=a(C)()`
`∀hasTopping.C`	`∀t (hasTopping(p, t) → C(t))`	`for_all(t, HasType(t, C))`
`¬ ⊓ ⊔`	`¬ ∧ ∨`	`not_(…)`, several conditions, `or_(…)`

(b) PREDICT **Some, only, none.** Mark with a cross which pizzas each query returns, and write each query as a DL concept. Then run Act 1(b).

```
Q1 = entity(p).where(for_all(t, HasType(t, VegetableTopping)))
Q2 = a(Pizza)(toppings=[a(MozzarellaTopping)(), a(TomatoTopping)()]).from_(pizzas)
Q3 = entity(p).where(not_(exists(t, HasType(t, CheeseTopping))))
```

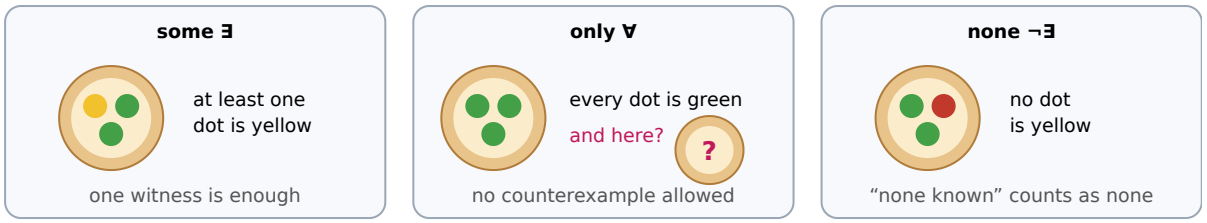


Figure 4. The three quantifier patterns of Worksheet 2, as the robot checks them on one pizza.

	margherita	napoletana	american_hot	mushroom	veggie	plain	unclosed	DL concept
Q1								
Q2								
Q3								

? Last week Protégé refused to classify `UnclosedPizza` (a pizza with some mozzarella) as vegetarian. **NOTEBOOK** Write the query `vegetarian` (no meat, no fish). Is unclosed_1 an answer? Who is right, the query or the reasoner?

(c) Following the relations. *Ada at table 1 does not like spicy food: “Is anything hot on my table?”* Write the orders that contain a pizza with a hot topping as a DL concept (roles hasPizza, hasTopping, hasSpiciness; the individual Hot). **NOTEBOOK** Write `hot_pizzas` and `hot_orders`.

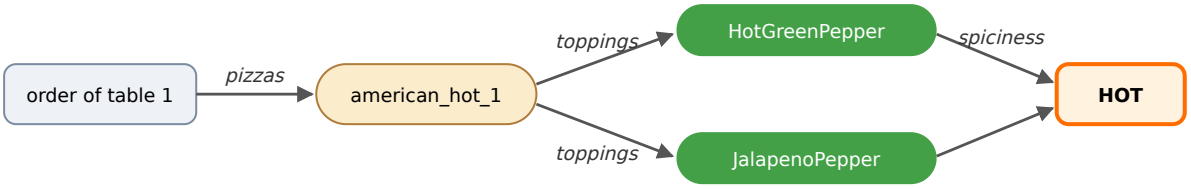


Figure 5. The path from Ada's order to a hot topping. There are two such paths.

PREDICT The notebook then asks for the witnesses themselves: `set_of(o.guest, ordered, topping).where(topping.spiciness == Spiciness.HOT)`. How many answers does it have, while `hot_orders` has one? Answer: _____

ACT 2 · DESCRIBING ≈ 12 min

“Bring me the pizza with olives.”

> *Commands do not name objects; they describe them. Ben at table 2 waves at the robot.*

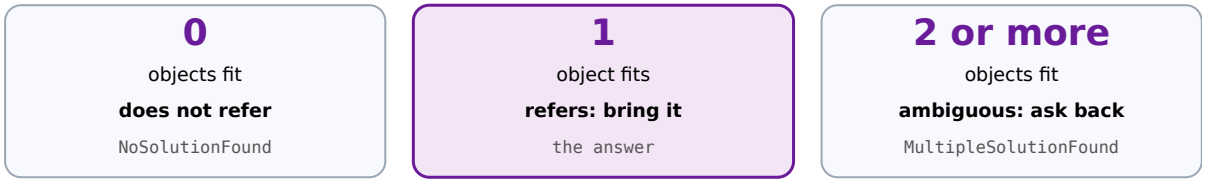


Figure 6. `a(…)` accepts any object that fits. `the(…)` claims that exactly one fits: a definite description.

(a) PREDICT ***a* or *the*?** For each command, how many pizzas fit, and what should the robot do? Then run Act 2(a).

Command	Pizzas that fit	What the robot does
"the pizza with ham"		
"the pizza with fish"		
"the pizza with olives"		

(b) Making a description precise. The robot asks back: "The one with fish, or the one without?" Ben: "Without fish." Write the precise description as a DL concept. NOTEBOOK Write it in EQL so that `the(…)` succeeds.

> ? "The pizza with cheese" does not refer tonight (five fit). Does "the pizza with cheese *of table 3*" refer? What does this tell you about `the`, and about how people resolve "the …" in everyday speech?

ACT 3 · CONSTRAINING ≈ 15 min

"Ben must never get milk."

> *Ben avoids milk. Nowhere does it say "napoletana_1 contains milk": it has to be computed. Mozzarella is a cheese, cheese contains milk, the pizza has that topping.*

(a) PAPER **Stored or computed?** In the notebook, `symbolic_function` turns a Python function into vocabulary of the query language, and a `Predicate` is a relation decided by a computation. For each relation below, is it stored in the objects (S) or computed (C)? NOTEBOOK Complete `allergens` and `IsSafeFor`.

hasTopping(x, y)	guest(o) = g	MozzarellaTopping ⊑ CheeseTopping	contains(x, milk)	IsSafeFor(x, g)

(b) PAPER **A constraint is a query that must have no answer.** The kitchen's rule is: every pizza of an order is safe for the guest of that order. Write it in first-order logic (use hasPizza, guest and IsSafeFor). A query cannot prove a ∀, but it can search for a counterexample: write that formula too.

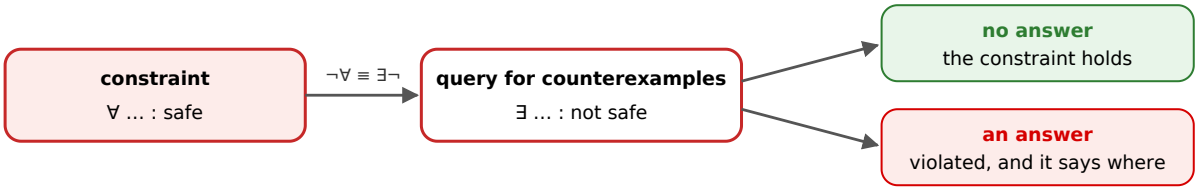


Figure 7. Checking a universal statement over one interpretation means looking for a counterexample.

PREDICT How many violations are there tonight? PEN Draw each one as a red line in Figure 1. NOTEBOOK Write `dangerous` and check.

> ? In OWL you could state the kitchen's rule as an axiom. If the facts violate it, what does the reasoner report? What does the query report instead?

ACT 4 · INFERRING ≈ 20 min

The kitchen writes warnings and labels.

> *The menu must list the allergens of every pizza and a label: vegetarian, vegan or meaty. Instead of computing these anew for every question, the kitchen writes them down once, as new facts.*

(a) A rule creates facts. In EQL, `inference(AllergenWarning)(pizza=D, …)` creates one new warning object for every pizza that fits the description D. The milk rule is

```
∀x (Pizza(x) ∧ ∃y (hasTopping(x, y) ∧ CheeseTopping(y)) → ∃w (Warning(w) ∧ about(w, x) ∧ allergen(w, milk)))
```

PAPER Which part of the formula is the description D, and which part does the rule build? PREDICT With a fish rule and a gluten rule (every pizza) added, how many warnings are there? _____

NOTEBOOK Write both rules.

(b) PREDICT Rules with exceptions. The chef's rule of thumb for the labels is a rule tree (Figure 9). Predict the label of each pizza, then run Act 4(b).

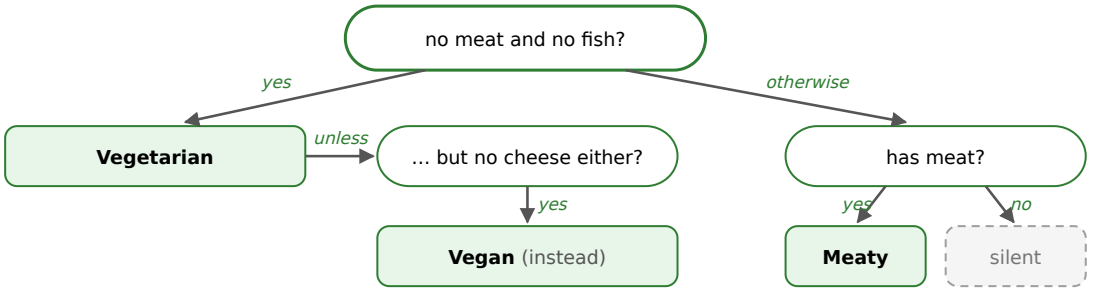


Figure 9. A rule tree: a default conclusion, an exception (*unless*, EQL: `refinement`) and an alternative (*otherwise*, EQL: `alternative`).

margherita	napoletana	american_hot	mushroom	veggie	plain	unclosed

(c) PREDICT The world moves on. *It is busy. The cook mixes up two tickets and grates parmesan over veggie_1, Ben's pizza.* What do the three sources say about veggie_1 now? Which of them protects Ben? Then run Act 4(c).

1. the constraint of Act 3, checked now	2. the milk warnings derived in (a)	3. the label of veggie_1, derived again

ACT 5 · GENERATING ≈ 13 min

"A pizza with mozzarella and tomato, please."

> *Dana sits down at table 4 and orders. Her order is a description, exactly like those of Act 1. What is new is who answers it.*

A query in EQL is a symbolic object; nothing is computed when it is written. A **backend** is whatever answers it. The same description can be handed to different backends:

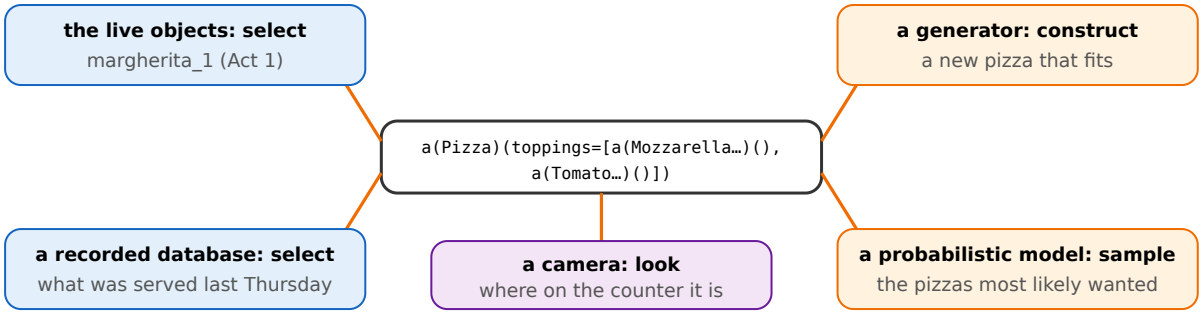


Figure 11. The description is written once, symbolically. Selecting, constructing, sampling and looking are different ways of answering it.

(a) PREDICT **Select or construct?** What does the counter answer, and what does the kitchen answer? Then Dana adds "… and a green pepper, I don't mind how hot" (`spiciness=...`, left open). How many pizzas fit that description? _____ NOTEBOOK Run Act 5.

> ? In Act 1, every answer was an object of the interpretation in Figure 1. What is an answer here? What does it mean, in the language of models, that Dana's open description has several answers?

(b) PAPER **One plan, many answerers.** The robot's next task is a plan with open slots, each written as a description:

Bring ① a pizza that is safe for Ben, lying ② at some pose on the counter, **to** ③ the table of Ben, **holding the plate with** ④ a grasp that works. **If none is on the counter:** ⑤ have one made.

Which kind of backend should answer each slot? Choose from **K** a query over the knowledge base, **R** rules and predicates, **L** looking with the camera, **S** sampling from a probabilistic model, **G** a generator.

① safe pizza	② its pose	③ Ben's table	④ grasp	⑤ have one made

EPILOGUE

The whole picture

≈ 5 min

Take home. One expression, a description, carried the whole evening. Its meaning is a concept, as in description logic; what you *do* with it decides whether it is a question, a reference, a constraint, the body of a rule or a specification. Next week: the same descriptions as the open parameters of robot plans.
To think about at home: a robot tidies a kitchen. Give one description of each of the five uses, with the backend that should answer it.

(c) PAPER **Your summary.** For each use of a description, give tonight's example and what it looks like in logic.

Use	The question behind it	Tonight's example	In logic
1 Ask	Which objects fit?		
2 Describe	Which one is meant?		
3 Constrain	Does anything violate it?		
4 Infer	What follows for each fit?		
5 Generate	What could fit?		